COMPUTER NETWORKS LABORATORY

EXPERIMENT 9

Capture and Analyze: HTTP and DNS Traffic

Capturing HTTP and DNS traffic using Wireshark and interpreting the header and payload of each captured packet.

Contact Hours: 02 **Filters Used:** dns / http

Introduction

Whenever we open a website, two things happen behind the scenes. First, our computer asks, "What is the IP address of this website name?" This step is handled by DNS. Second, our computer asks the web server, "Please give me this page." This step is handled by HTTP.

Both DNS and HTTP are application-layer protocols. They do not travel alone; they are carried by the transport-layer protocols studied in Experiment 8 (UDP and TCP). Capturing and studying this traffic helps us understand exactly how a web page request works, from name resolution to page delivery.

Simple analogy: DNS is like searching a friend's name in your phone contacts to get their number. HTTP is like actually calling that number and asking for something.

Application Protocol	Carried By (Transport)	Default Port
DNS	UDP (TCP for large responses)	53
HTTP	TCP	80

Aim

To capture HTTP and DNS traffic using Wireshark and interpret the header and payload of each captured packet.

In simple words, the aim of this practical is to "see" what really happens on the network when we open a website. Instead of only reading about DNS and HTTP in books, we will record the actual packets sent and received by our computer and study them field by field.

Objectives

This practical involves the following steps and objectives:

- **Capturing DNS traffic:** generate a DNS lookup (for example using nslookup) and record the query and response packets exchanged with the DNS server.
- **Capturing HTTP traffic:** visit a plain-HTTP website and record the request sent by the browser and the response sent by the web server, along with the TCP handshake that precedes it.
- **Identifying their respective headers:** locate and understand the header fields of each protocol, i.e. the 12-byte DNS header (Transaction ID, Flags, counts) and the HTTP request/status line and header lines (Host, User-Agent, Content-Type, etc.).
- **Examining their payloads:** find the actual data carried by the packets, i.e. the domain name and IP address in DNS, and the HTML content of the web page in HTTP.
- **Comparing the two protocols:** compare DNS and HTTP in terms of purpose, transport protocol, port number, connection setup, message size and readability.

Learning Outcomes

After completing this practical, we should be able to:

- Explain how a web page request works step by step: DNS resolution, TCP connection, HTTP request and HTTP response.
- Use Wireshark to start a capture, apply display filters such as dns and http, and inspect individual packets.
- Distinguish between the header and the payload of a packet, and read each layer (Ethernet, IP, TCP/UDP, application) in the packet detail pane.
- Explain why DNS normally uses UDP port 53 while HTTP uses TCP port 80.
- Understand why unencrypted protocols expose their contents, and why secure alternatives like HTTPS and DNS over HTTPS are important.

Scope of the Practical

The capture is done on our own computer's active network interface (Wi-Fi or Ethernet) for a short period of time. Only plain-HTTP traffic is analysed for its payload, because HTTPS traffic is encrypted and its content cannot be read. DNS queries are analysed using the standard UDP port 53 communication.

Tools & Requirements

The following tools and requirements are needed to perform this practical.

Hardware Requirements

- A computer or laptop (Windows, Linux or macOS) with enough permission to install software and capture packets.
- A working network adapter: Wi-Fi or Ethernet. Wireshark captures traffic on whichever interface is selected.
- An active internet connection, so that real DNS lookups and website requests can be generated.

Software Requirements

Tool	Purpose in this practical
Wireshark	Captures live packets and displays them in a readable, layer-by-layer form. Used to apply the dns and http filters and inspect every field.
Npcap / libpcap (capture driver)	Installed along with Wireshark. It lets Wireshark read packets from the network interface (Npcap on Windows, libpcap on Linux/macOS).
Web browser (Chrome, Firefox, Edge)	Used to open a plain-HTTP test website such as http://neverssl.com or http://example.com, which generates HTTP traffic.
Command Prompt / Terminal	Used to run nslookup www.google.com to generate a DNS query, and ipconfig /flushdns to clear the DNS cache (Windows).

Tool	Purpose in this practical
nslookup (command-line utility)	Sends a DNS query to the configured DNS server on demand, so that a clear query-response pair appears in the capture.

Network and Test-Site Requirements

- A plain-HTTP (not HTTPS) test website, e.g. neverssl.com or example.com, because HTTPS encrypts the content and nothing readable would appear under the http filter.
- Access to a DNS server (automatically provided by the router or ISP, or a public one such as 8.8.8.8).
- Ability to flush the DNS cache before the experiment, so that the lookup is not answered from the local cache.

Prerequisite Knowledge

- Basic knowledge of the OSI and TCP/IP models, especially the application and transport layers.
- The working of TCP (three-way handshake, sequence and acknowledgment numbers) and UDP (connectionless, 8-byte header), as studied in Experiment 8.
- Meaning of IP address, MAC address and port number.
- Basic idea of a client–server model.

About Wireshark

Wireshark is a widely used, free and open-source network protocol analyzer. It captures packets passing through a selected network interface and displays them in a structured form, from the Ethernet layer up to the application layer, so that every header field and payload can be inspected directly. This makes it an essential learning tool for understanding how network protocols actually behave.

The Wireshark window has three main panes:

Pane	What it shows
Packet List	One row per captured packet: number, time, source, destination, protocol, length and a short info summary. Packets can be filtered and selected here.
Packet Details	The selected packet broken into layers (Frame, Ethernet, IP, TCP/UDP, DNS/HTTP). Each layer can be expanded to see individual header fields.
Packet Bytes	The raw bytes of the packet in hexadecimal with the matching ASCII text on the side. Readable payload such as a domain name or HTML text can be seen here.

Wireshark features used in this practical:

- **Capture interface selection:** choosing the Wi-Fi or Ethernet adapter on which the traffic is generated.
- **Display filters:** dns, http, http.request, http.response and dns.qry.name == "example.com" to show only the traffic of interest.
- **Follow TCP Stream:** rebuilds the complete HTTP request and response conversation in readable text.
- **Colouring rules:** Wireshark colours packets by protocol, which helps in quickly spotting DNS and HTTP traffic.
- **Save capture:** the recorded session can be saved as a .pcapng file and reopened later for analysis.

Theory

1. What is DNS?

DNS (Domain Name System) converts human-readable names such as www.google.com into IP addresses. It is often called the “phonebook of the Internet”, because people remember names while computers communicate using numbers.

Key points:

- DNS uses UDP port 53 for normal queries, since they are small and no handshake is needed.
- TCP port 53 is used only for large responses or zone transfers.
- It works on a query → response model.
- Answers are cached by the computer, so repeated lookups may not appear in a capture.

DNS message structure:

Section	Meaning
Header (12 bytes)	Transaction ID, flags, and counts of questions and answers
Question	The name being asked about and the record type
Answer	The IP address or record that is returned
Authority / Additional	Extra information about name servers

Important DNS header fields:

- **Transaction ID:** matches a response to its query.
- **Flags:** QR (0 = query, 1 = response), Opcode, AA, TC, RD (recursion desired), RA (recursion available), Rcode (response code).
- **Questions / Answer RRs / Authority RRs / Additional RRs:** counts of each section.

Common DNS record types:

Type	Purpose
A	Hostname to IPv4 address
AAAA	Hostname to IPv6 address
CNAME	Alias (canonical name)
PTR	IP address to hostname (reverse lookup)
MX	Mail server for a domain

2. What is HTTP?

HTTP (HyperText Transfer Protocol) is the protocol used by a web browser to request web pages and other resources from a web server.

Key points:

- HTTP uses TCP port 80, so a TCP three-way handshake (SYN, SYN-ACK, ACK) takes place first.
- It follows a request → response model.
- It is stateless: each request is independent of the previous one.
- It is plain text, so it can be read in Wireshark. HTTPS (port 443) is the encrypted version, whose contents cannot be read.

HTTP request structure:

```
GET /index.html HTTP/1.1        <- Request line (method, path, version)
Host: example.com               <- Headers
User-Agent: Mozilla/5.0
Accept: text/html
                                <- Blank line
(optional body)                 <- Payload
```

HTTP response structure:

```
HTTP/1.1 200 OK                  <- Status line
Content-Type: text/html          <- Headers
Content-Length: 1256
                                 <- Blank line
<html> ... </html>               <- Payload (the web page)
```

Common methods: GET (fetch data), POST (send data), PUT, DELETE, HEAD.

Common status codes:

Code	Meaning
200	OK – request successful
301 / 302	Redirect – resource moved
304	Not Modified – cached copy can be used
404	Not Found

Code	Meaning
500	Internal Server Error

3. Header vs Payload

Every packet has a header, which carries control information, and a payload, which carries the actual data. Each packet is wrapped in layers: Ethernet → IP → TCP/UDP → HTTP/DNS. In Wireshark's detail pane, each of these layers can be expanded and studied.

	HTTP	DNS
Header	Request/status line plus headers (Host, User-Agent, Content-Type, etc.)	12-byte header (ID, flags, counts)
Payload	Body, such as the HTML page, image or form data	Question and answer records (domain name, IP address)

4. DNS vs HTTP Comparison

Feature	DNS	HTTP
Purpose	Name to IP resolution	Transfer web content
Transport protocol	UDP (mostly)	TCP
Port	53	80
Connection setup	None	Three-way handshake
Model	Query / Response	Request / Response
Message size	Small	Can be large
Readability	Readable (unencrypted)	Readable (if not HTTPS)

Capture Method

1. Open Wireshark and select the active interface (Wi-Fi or Ethernet).
2. Start the capture.
3. Generate DNS traffic by running in Command Prompt: nslookup www.google.com
4. Generate HTTP traffic by opening a plain HTTP (not HTTPS) site in a browser, e.g. http://neverssl.com or http://example.com.
5. Stop the capture.
6. Apply the display filters dns and http.
7. Select a packet and inspect it in the detail pane.

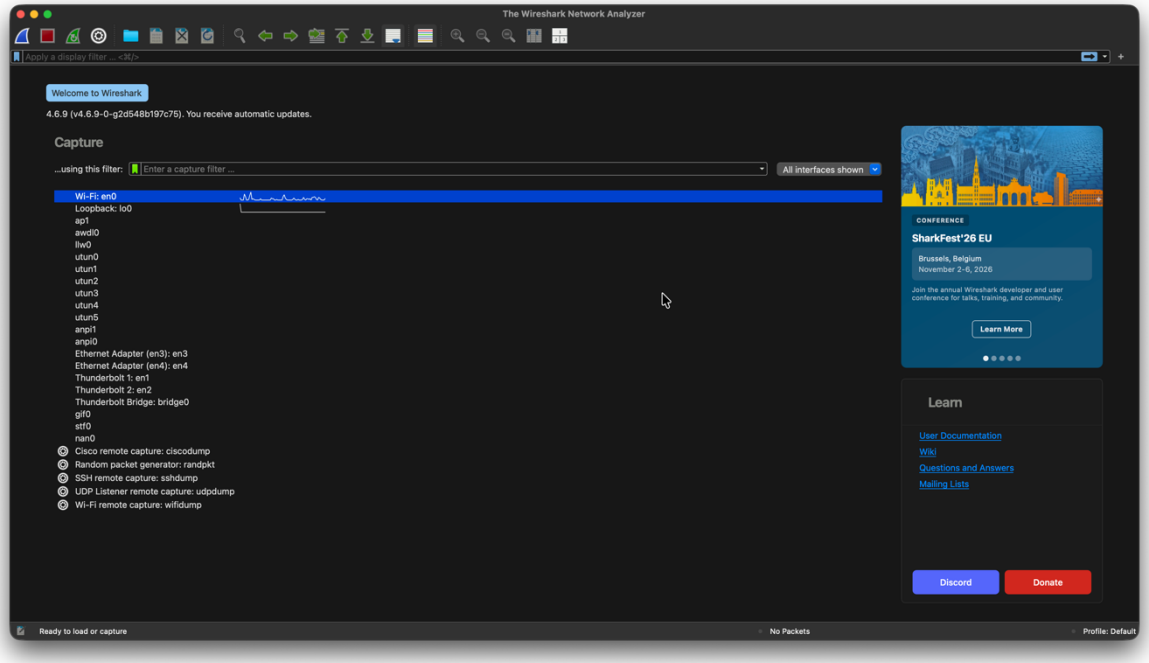
Useful Wireshark display filters:

Filter	Shows
dns	Only DNS packets
http	Only HTTP packets
http.request	Only HTTP requests
http.response	Only HTTP responses
dns.qry.name == "example.com"	DNS packets for one specific domain

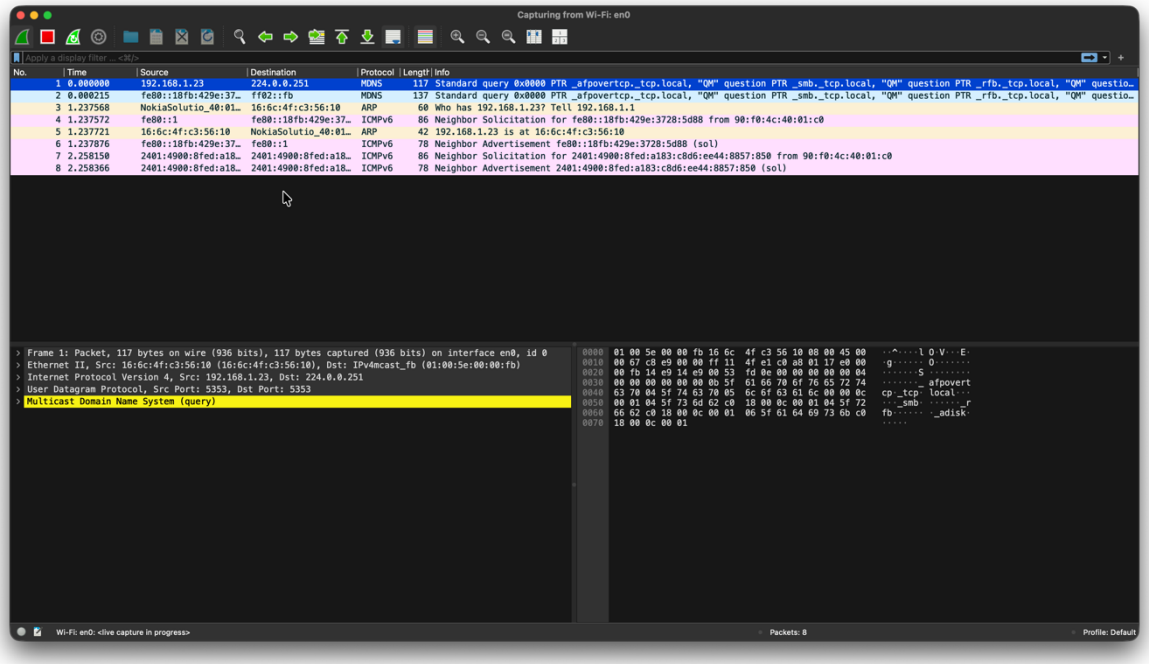
Note: Most modern websites use HTTPS, so their content is encrypted and HTTP packets will not appear. A plain-HTTP test site such as neverssl.com should be used. Also run ipconfig /flushdns (Windows) before the DNS lookup, otherwise cached results may not generate a DNS query.

Steps / Procedure

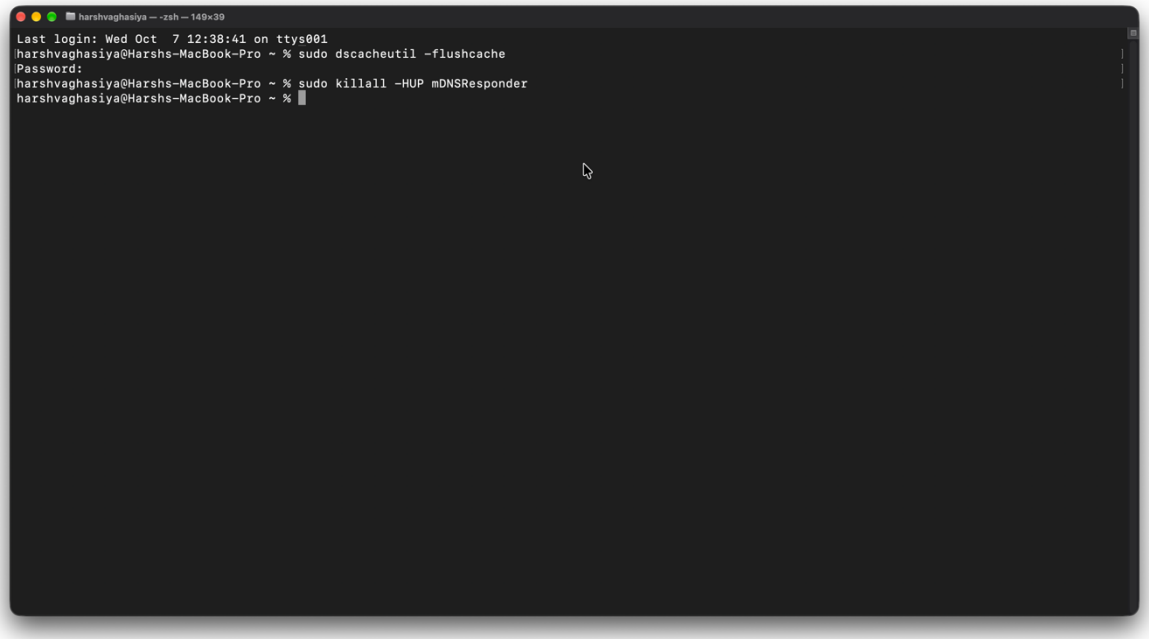
1. Install and open Wireshark.
2. Select the active network interface.



3. Start the packet capture.



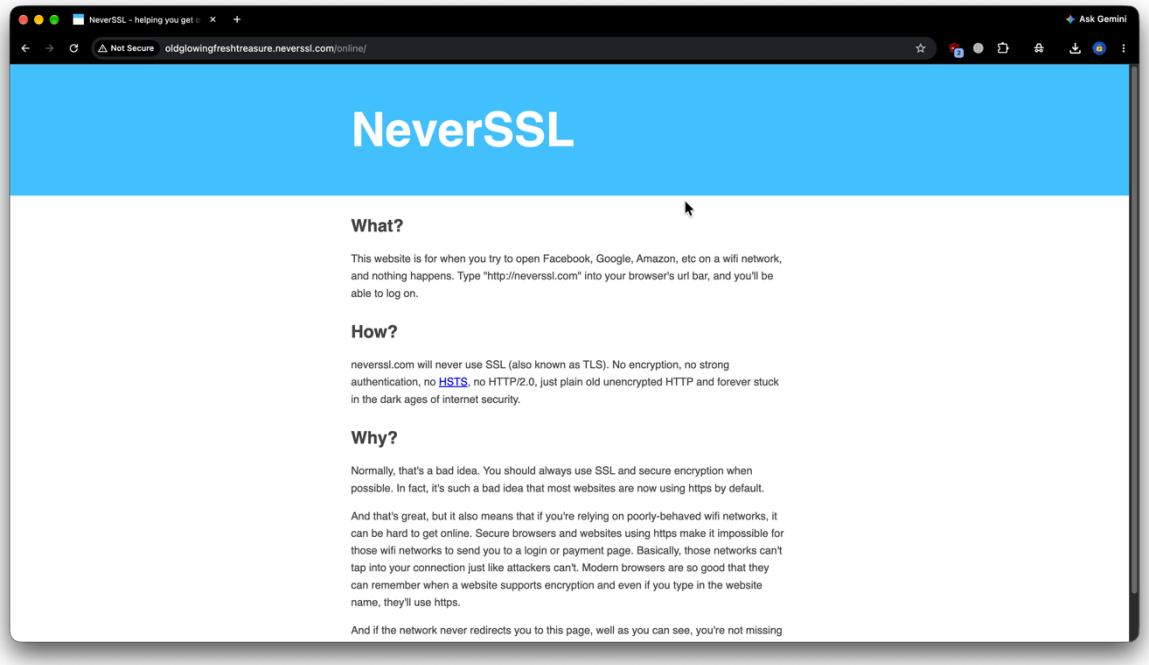
4. Flush the DNS cache using ipconfig /flushdns.
   For windows (CMD): ipconfig /flushdns



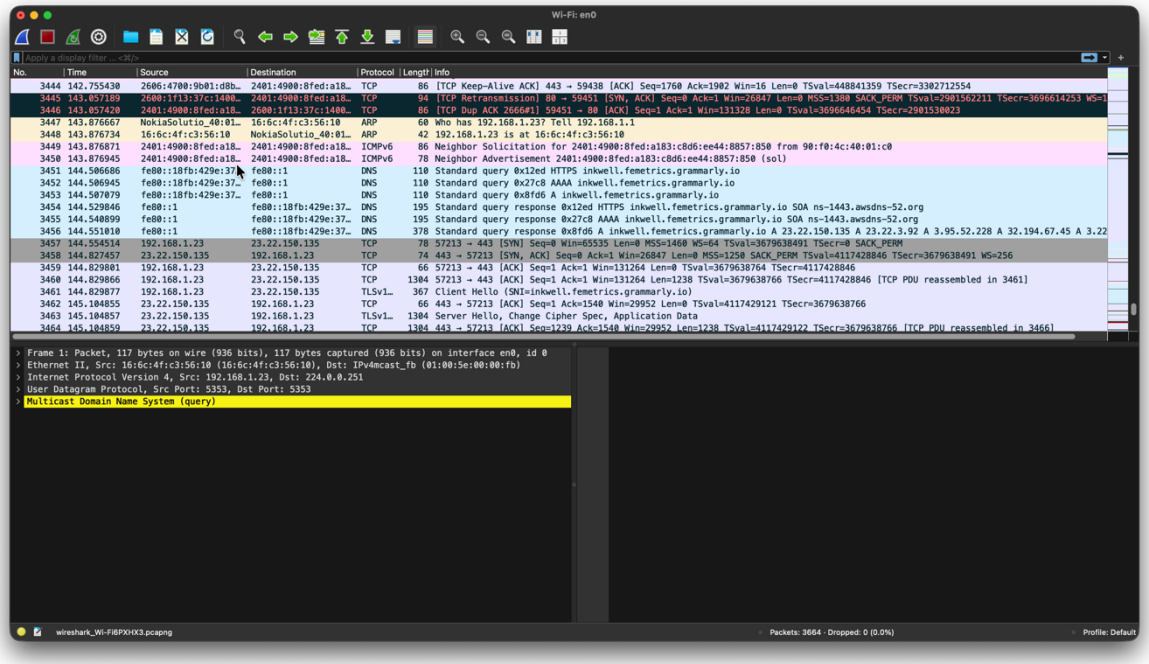
5. Run nslookup www.google.com to generate DNS traffic.
   run in cmd : nslookup www.google.com



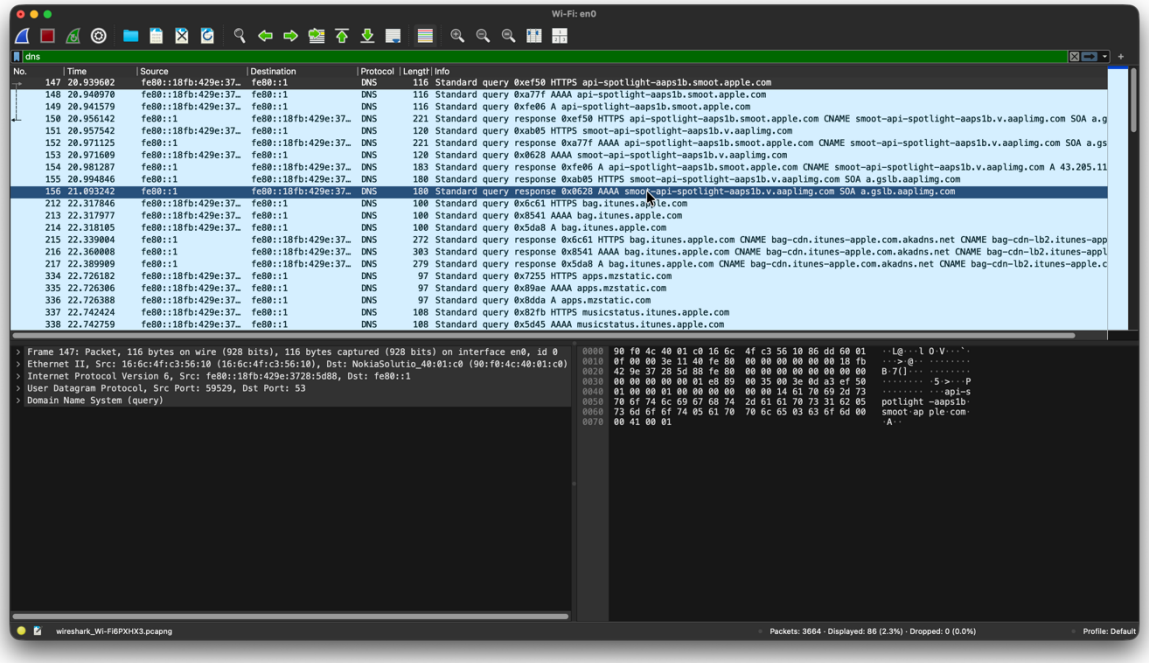
6. Open http://neverssl.com in a browser to generate HTTP traffic.



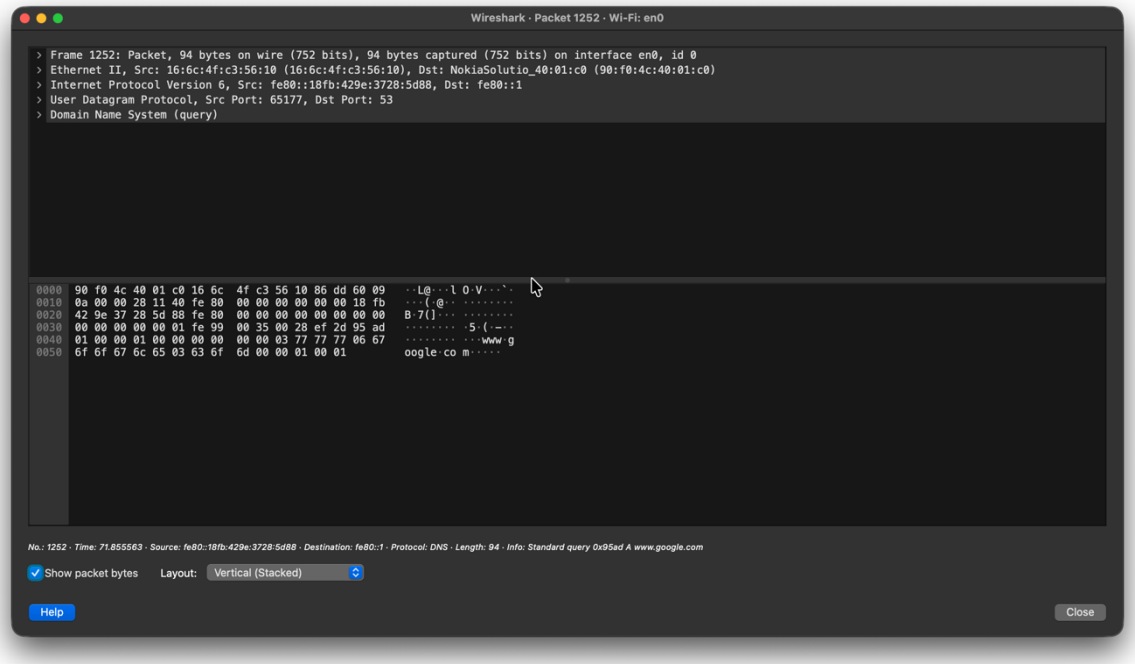
7. Stop the capture.



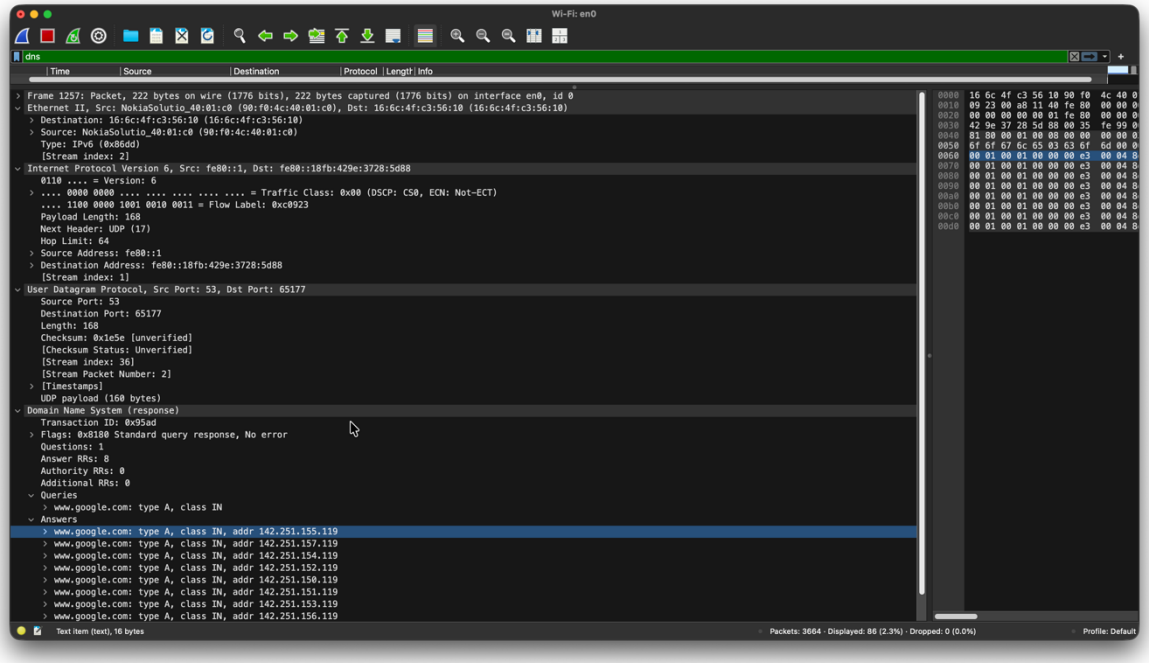
8. Apply the dns filter and locate a DNS query and its response.



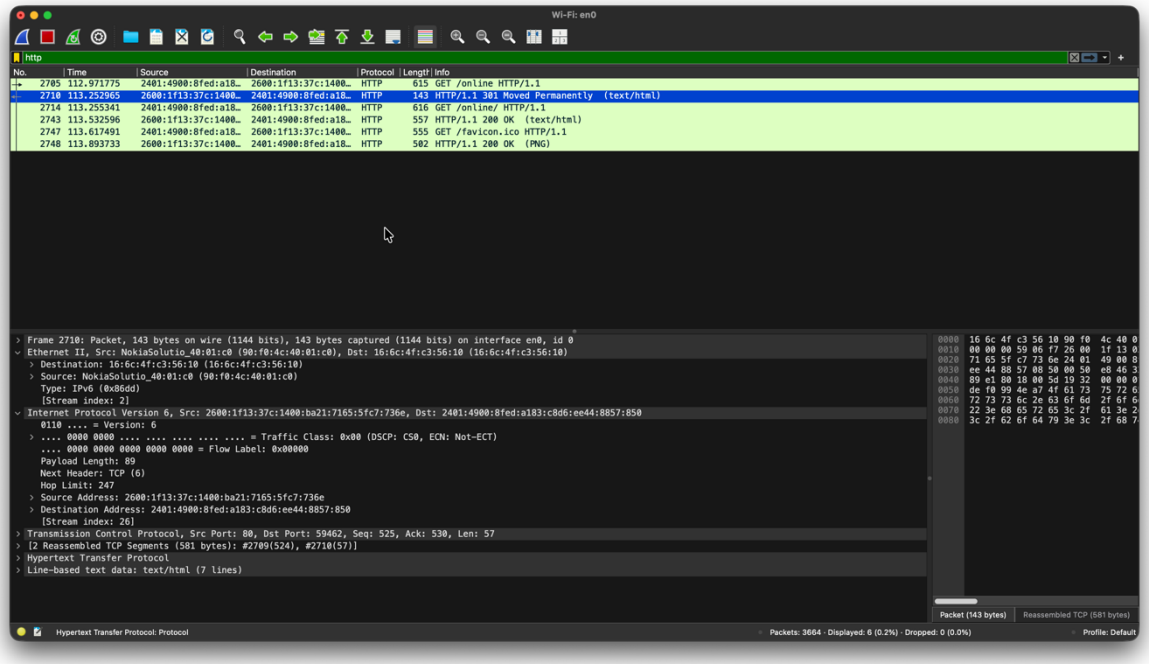
9. Select the DNS query. Expand User Datagram Protocol and Domain Name System (query), and note the fields.



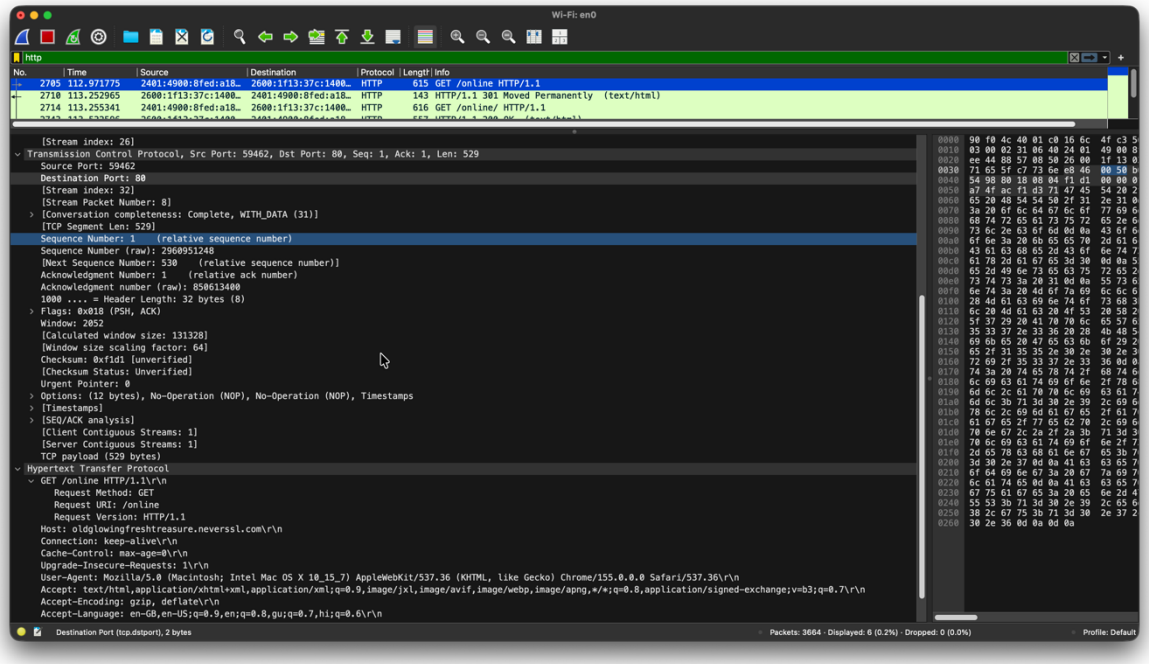
10. Select the DNS response and note the answer section (IP address returned).



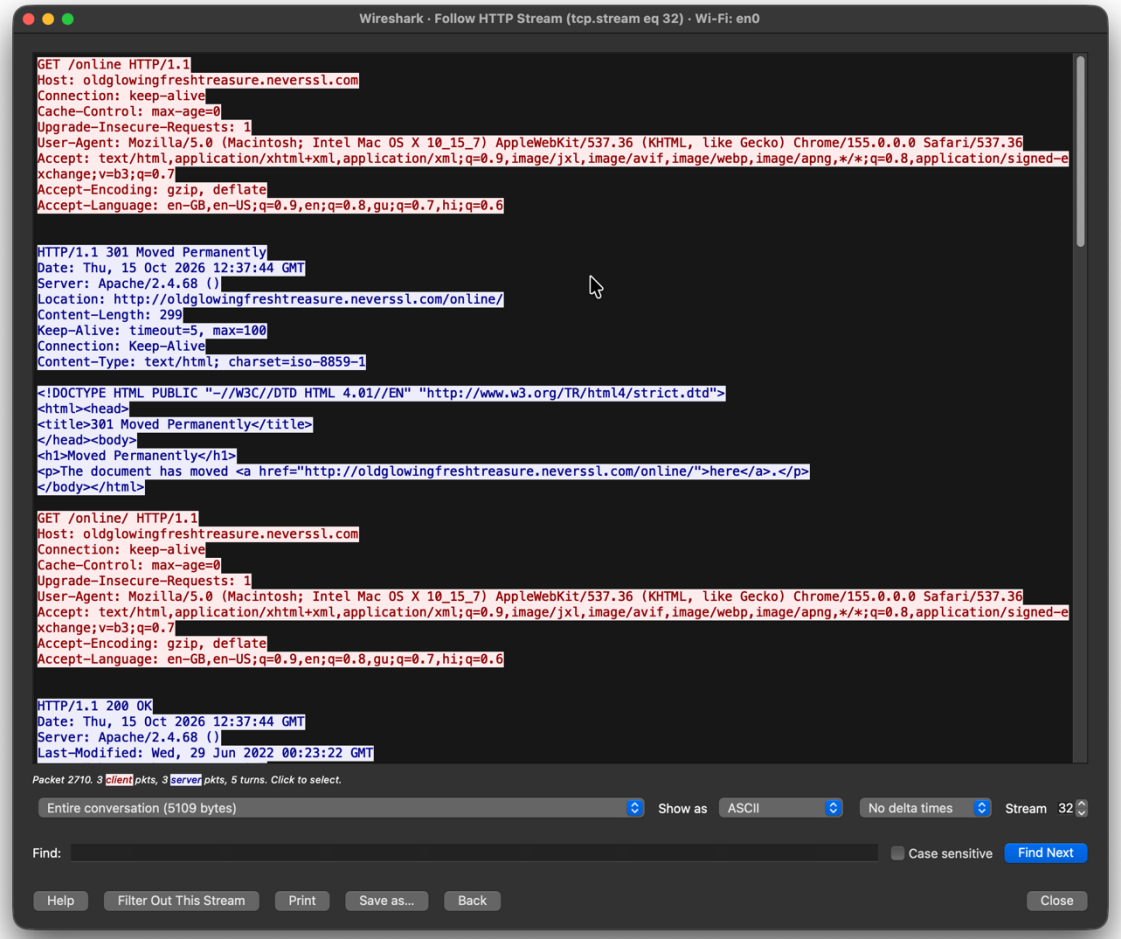
11. Apply the http filter and locate the GET request and the HTTP/1.1 200 OK response.



12. Expand Hypertext Transfer Protocol and note the request line, headers and status code.



13. Right-click an HTTP packet and choose Follow → TCP Stream to view the full request and response in readable text.



14. Record the observations in the tables below.
15. Save the capture file, if required.

Result / Observation

DNS Result

For the captured DNS packets, the source and destination IP addresses, ports, transport protocol, DNS header fields, question and answer records were observed and recorded.

Observation Table – DNS (replace with your actual captured values)

Field	Observed Value
Source IP	192.168.x.x (your PC)
Destination IP	8.8.8.8 (DNS server)
Source Port	Random, e.g. 58173
Destination Port	53

Field	Observed Value
Transport Protocol	UDP
Transaction ID	0x983d
Flags (Query)	0x0100 (Standard query, recursion desired)
Questions	1
Query Name / Type	www.google.com / A
Flags (Response)	0x8180 (Response, no error)
Answer	IP address returned, e.g. 142.250.x.x
Payload Present	Yes (question and answer records)

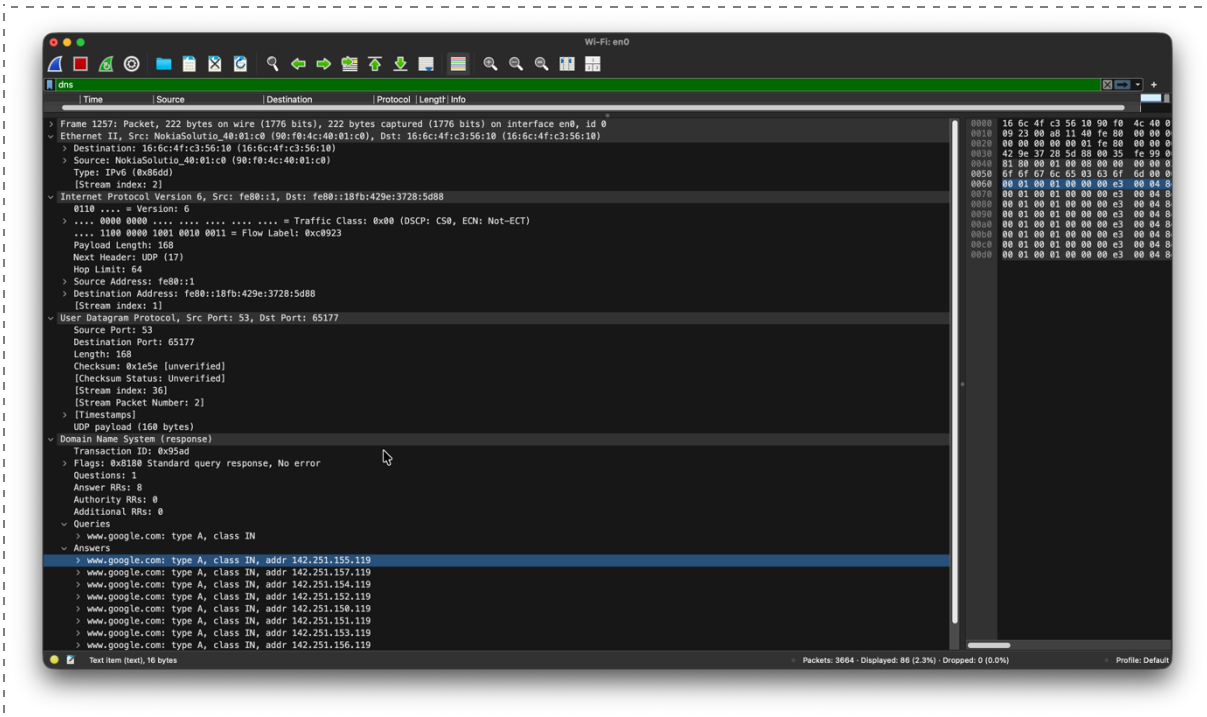


Fig. R1 — Captured DNS query and response over UDP port 53.

Observations:

- The DNS query travelled over UDP with destination port 53.
- There was no handshake; the query was sent directly.
- The response carried the same Transaction ID as the query.
- The response contained the IP address of the requested domain in its answer section.

HTTP Result

For the captured HTTP packets, the IP addresses, ports, request method, headers, status code and payload were observed and recorded.

Observation Table – HTTP (replace with your actual captured values)

Field	Observed Value
Source IP	192.168.x.x (your PC)
Destination IP	Web server IP
Source Port	Random, e.g. 51517
Destination Port	80
Transport Protocol	TCP
Request Method	GET
Request URI	/
HTTP Version	HTTP/1.1
Host Header	neverssl.com
User-Agent	Browser details (e.g. Mozilla/5.0 ...)
Response Status	200 OK
Content-Type	text/html
Payload Present	Yes (HTML content of the page)

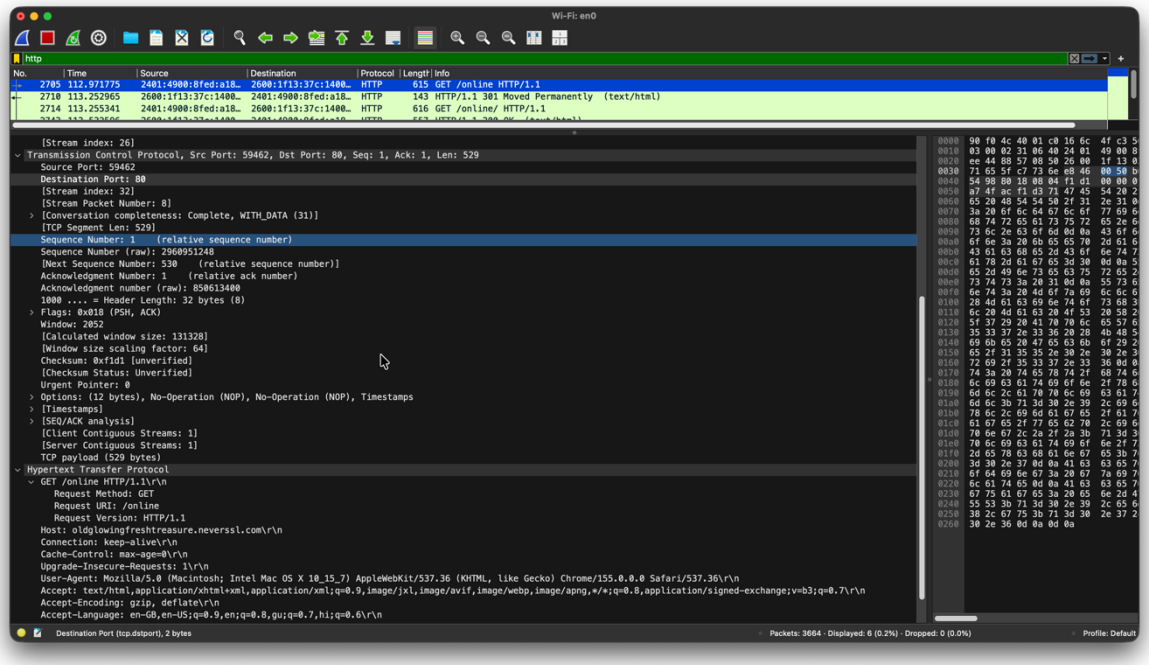


Fig. R2 — Captured HTTP request and response over TCP port 80.

Observations:

- A TCP three-way handshake (SYN, SYN-ACK, ACK) took place before the HTTP request.
- The client sent a GET request and the server replied with 200 OK, carrying the web page in the payload.
- HTTP headers and payload were readable in plain text, since HTTP is not encrypted.

- The HTTP header carried control information, while the payload carried the actual HTML.

Conclusion

In this practical, HTTP and DNS traffic was successfully captured and analyzed using Wireshark. The headers of both protocols were identified and analyzed, and their payloads were examined.

It was observed that DNS uses UDP (port 53) for fast, lightweight query-response communication to resolve a domain name into an IP address, whereas HTTP uses TCP (port 80), which first establishes a connection and then transfers web content reliably through request and response messages.

The practical also showed how application-layer protocols depend on transport-layer protocols, and that unencrypted protocols like HTTP and DNS expose their contents in captured packets. This highlights the need for secure alternatives such as HTTPS and DNS over HTTPS.